

HBAExpress

Cahier des charges - Infrastructure & Déploiement

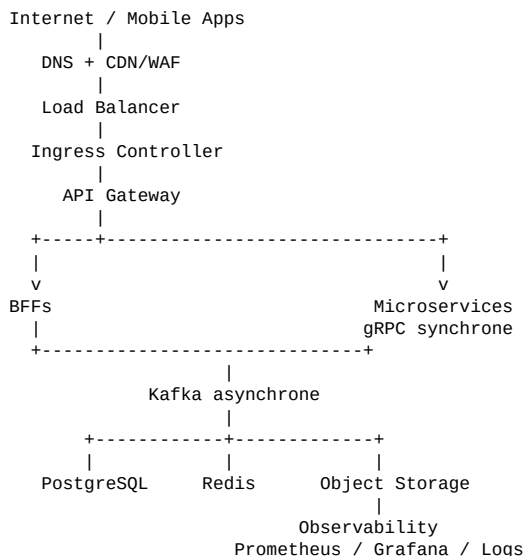
Marketplace + Food + Delivery | Microservices | Kubernetes | gRPC | Kafka

Objectif : définir une infrastructure de production sécurisée, hautement disponible, observable et automatisable pour les services HBAExpress. Le présent document complète le cahier des charges Backend et formalise les environnements, Kubernetes, réseau, stockage, Kafka, bases de données, sécurité, CI/CD, sauvegardes, observabilité, déploiements et procédures d'exploitation.

Version 1.0 - Août 2026

1. Principes d'architecture

L'infrastructure doit être conçue pour permettre une montée en charge progressive : démarrage maîtrisé sur un nombre limité de nœuds, puis ajout horizontal de capacité sans refonte des applications. Les microservices restent stateless autant que possible ; les états durables sont placés dans PostgreSQL, Redis, le stockage objet et Kafka.



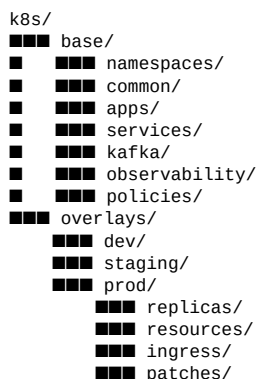
Règle réseau : seuls l'Ingress/API Gateway et les endpoints explicitement publics sont exposés à Internet. PostgreSQL, Redis, Kafka et les ports gRPC restent sur le réseau privé du cluster.

2. Environnements

Environnement	Usage	Données	Déploiement
Local	Développement individuel	Fixtures / données synthétiques	Docker Compose ou Kubernetes local
Development	Intégration continue des branches	Données non sensibles	Automatique
Staging	Validation pré-production, E2E, charge	Données anonymisées	Automatique après merge
Production	Utilisateurs réels	Données réelles	Promotion contrôlée + rollback

Chaque environnement possède ses propres namespaces Kubernetes, secrets, bases, topics Kafka et buckets. Aucun secret ni accès production ne doit être réutilisé en staging.

3. Topologie Kubernetes cible



Le cluster doit utiliser des **Deployments** pour les workloads stateless, des **StatefulSets** uniquement lorsque nécessaire, des **Services ClusterIP** pour les communications internes et un Ingress Controller pour le trafic HTTP externe.

4. Dimensionnement initial recommandé

Composant	Staging	Production initiale	Évolution
Control plane	Managé ou 1 nœud dédié	Managé recommandé	HA fournisseur

Composant	Staging	Production initiale	Évolution
Worker nodes	2 x 4 vCPU / 8 Go	3 x 8 vCPU / 16 Go	Autoscaling horizontal
API Gateway/BFF	1 replica	2-3 replicas	HPA CPU/RPS
Microservices critiques	1	2-3	HPA + PDB
Microservices secondaires	1	1-2	HPA selon métriques
PostgreSQL	instance séparée	DB managée ou nœud dédié + réplica	Read replicas / HA
Kafka	1 broker dev	3 brokers production	Ajout brokers/partitions
Redis	1 instance	Primary + replica/Sentinel ou managé	Cluster si nécessaire

Ces valeurs sont un point de départ et non un dimensionnement définitif. Les ressources réelles doivent être ajustées à partir des métriques CPU, mémoire, RPS, latence gRPC, consumer lag Kafka et connexions DB.

5. Réseau et exposition

Flux	Port	Exposition	Règle
HTTPS client -> Ingress	443	Public	TLS obligatoire
HTTP	80	Public	Redirection vers HTTPS
BFF -> services gRPC	50051+ interne	Privé	mTLS recommandé
Kafka	9092/9093	Privé	SASL/TLS en production
PostgreSQL	5432	Privé	NetworkPolicy + TLS
Redis	6379	Privé	ACL/TLS
Prometheus/Grafana	interne	Privé/VPN	SSO/RBAC

Des Kubernetes NetworkPolicies doivent appliquer un modèle **deny by default** : chaque service ne peut contacter que ses dépendances explicitement déclarées.

6. DNS, TLS, WAF et Gateway

Les domaines publics sont résolus via un fournisseur DNS. Un CDN/WAF peut être placé devant l'Ingress pour filtrer DDoS, bots et requêtes malveillantes. Les certificats TLS sont renouvelés automatiquement via cert-manager/ACME. L'API Gateway applique authentification, validation de token, quotas, rate limiting, CORS, taille maximale des requêtes et propagation des identifiants de corrélation.

```
api.hba-express.*    -> API Gateway
admin.hba-express.*  -> Admin Web
grafana.internal.*   -> accès privé/VPN
argocd.internal.*     -> accès privé/VPN
```

7. Déploiement des microservices

Objet	Exigence
Deployment	rolling update, replicas >= 2 pour les services critiques
Readiness probe	le pod ne reçoit aucun trafic avant disponibilité réelle
Liveness probe	redémarrage uniquement en cas de blocage réel
Startup probe	pour les services au démarrage lent
Resources	requests + limits obligatoires
PDB	empêcher l'arrêt simultané des replicas critiques
HPA	CPU/mémoire puis métriques applicatives
SecurityContext	non-root, filesystem read-only lorsque possible

```
resources:
  requests:
    cpu: "250m"
    memory: "256Mi"
  limits:
    cpu: "1000m"
    memory: "768Mi"

strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 0
    maxSurge: 1
```

8. gRPC en production

Toutes les communications synchrones inter-services utilisent gRPC. Les services sont découverts via DNS Kubernetes. Les appels doivent propager correlationId/traceparent, appliquer deadline/timeout, retry uniquement sur erreurs transitoires et circuit breaker. Aucun retry automatique ne doit dupliquer une commande non idempotente.

Paramètre	Standard
Deadline	obligatoire sur tous les appels
Retry	UNAVAILABLE / RESOURCE_EXHAUSTED selon politique

Paramètre	Standard
Keepalive	configuré pour connexions longues
Load balancing	Service Kubernetes ou service mesh
Sécurité	mTLS inter-services recommandé
Contrats	Protobuf versionné dans package partagé/registry

9. Kafka

Kafka constitue le bus événementiel asynchrone. La production utilise au minimum trois brokers lorsque Kafka est auto-hébergé. Les topics sont versionnés, partitionnés par agrégat et répliqués. Les producers utilisent idempotence et acks=all. Les consumers utilisent consumer groups, Inbox/idempotence et DLQ conformément au cahier des charges Backend.

Élément	Production
Replication factor	3
min.insync.replicas	2
Producer acks	all
Producer idempotence	activée
Compression	lz4 ou zstd
Consumer lag	monitoré + alerting
Schema	versionné; compatibilité contrôlée
DLQ	topic dédié par famille d'événements
Outbox	obligatoire pour publication liée à une transaction métier

10. Bases de données

Le principe retenu est **database ownership per service** : un microservice est seul propriétaire de son schéma/données et les autres services n'accèdent jamais directement à ses tables. PostgreSQL est recommandé pour les données transactionnelles. Redis est réservé au cache, rate limiting, sessions temporaires et verrous distribués lorsque nécessaires.

Domaine	Stockage principal	Compléments
Identity/User	PostgreSQL	Redis pour sessions/OTP
Marketplace	PostgreSQL	Redis cache; moteur de recherche optionnel
Food	PostgreSQL	Redis disponibilité/cache
Payment/Wallet	PostgreSQL	aucune perte tolérée; audit
Delivery	PostgreSQL	Redis pour positions/états temps réel
Media	Métadonnées PostgreSQL	S3/MinIO pour objets

11. Stockage objet

Images produits, photos restaurants, justificatifs et preuves de livraison sont stockés dans un stockage compatible S3. Les buckets ne sont pas publics par défaut. L'accès se fait via URL signée à durée limitée ou via CDN pour les médias publics validés. Versioning et lifecycle policies sont activés selon le type de données.

12. Secrets et configuration

Aucun secret ne doit être stocké dans Git, Dockerfile, image ou ConfigMap. Les secrets sont injectés au runtime depuis un gestionnaire de secrets (Vault, secret manager cloud ou mécanisme équivalent). Les clés de paiement, credentials Kafka, mots de passe DB, clés JWT et certificats font l'objet d'une rotation.

```
Configuration non sensible -> ConfigMap
Secret                    -> Secret Manager / Vault
Certificat                -> cert-manager / secret TLS
Variables par environnement -> overlays / Helm values
Jamais : .env de production commité dans Git
```

13. CI/CD

```
Pull Request
|
+--> lint / format
+--> unit tests
+--> SAST / dependency scan
+--> build
+--> integration tests
|
Merge main
|
+--> build image OCI
+--> SBOM
+--> vulnerability scan
+--> sign image
+--> push registry
|
+--> deploy staging
+--> smoke + E2E
|
Approval / promotion
|
+--> deploy production
+--> health checks
+--> metrics verification
+--> rollback automatique si échec
```

Les images sont immuables et identifiées par SHA/semver ; le tag latest n'est jamais utilisé pour un déploiement production reproductible.

14. Stratégie de release

Stratégie	Usage
Rolling update	défaut pour services stateless
Blue/Green	Gateway/BFF ou changements à risque élevé
Canary	nouvelle version exposée progressivement à un pourcentage de trafic
Feature flags	découpler déploiement technique et activation fonctionnelle

Le rollback doit être automatisable vers l'image précédente. Les migrations DB doivent suivre une stratégie backward-compatible : expand -> deploy -> migrate -> contract.

15. Migrations de base de données

1. Ajouter colonnes/tables compatibles
2. Déployer code compatible ancien + nouveau schéma
3. Migrer/backfill les données
4. Vérifier métriques et intégrité
5. Basculer les lecteurs/écrivains
6. Supprimer ancien schéma lors d'une release ultérieure

Une migration destructive ne doit jamais être exécutée avant que toutes les versions encore actives des services puissent fonctionner sans l'ancien champ.

16. Observabilité

Signal	Outil / exigence
Metrics	Prometheus; Grafana dashboards
Logs	logs JSON centralisés; Loki/ELK/OpenSearch
Traces	OpenTelemetry; Tempo/Jaeger
Alerting	Alertmanager + canal d'astreinte
Correlation	traceId + correlationId dans REST, gRPC, Kafka et logs
Audit	journal séparé pour opérations sensibles

Dashboards minimum : disponibilité API, p50/p95/p99, taux 5xx, saturation CPU/mémoire, pods restart, connexions DB, pool gRPC, Kafka consumer lag, taux d'échec paiement, durée checkout, dispatch delivery et erreurs de notifications.

17. SLO et alertes

Indicateur	Cible initiale
Disponibilité API critique	99,9 % mensuel
Latence API p95	< 500 ms hors fournisseurs externes
Latence gRPC p95	< 150 ms sur appels internes courants
Erreur serveur	< 1 %
Kafka consumer lag	alerte selon volume/âge du message
RPO données transactionnelles	<= 15 min
RTO service critique	<= 60 min

18. Sauvegarde et Disaster Recovery

Actif	Sauvegarde	Rétention minimale	Test restauration
PostgreSQL	snapshots + WAL/PITR	30 jours	mensuel
Kafka	réplication + stratégie selon criticité	selon topics	trimestriel
Object storage	versioning + réplication si possible	selon politique	trimestriel
Secrets/config	backup chiffré / IaC reproductible	historique	trimestriel
Kubernetes	GitOps/IaC = source de vérité	Git	reconstruction testée

Une sauvegarde n'est considérée valide qu'après un test de restauration. Le plan DR documente perte d'un pod, d'un nœud, de la DB primaire, du cluster et d'une région/fournisseur.

19. Sécurité infrastructure

- RBAC Kubernetes avec principe du moindre privilège.
- ServiceAccounts distincts par workload.
- Pods non-root; capacités Linux minimales; seccomp activé.
- NetworkPolicies deny-by-default.
- Images scannées avant promotion production.
- Registry privé et images signées.
- Secrets chiffrés au repos et en transit.
- Accès administrateur via VPN/bastion/SSO + MFA.
- SSH direct limité; authentification par clés, jamais par mot de passe.
- Audit des opérations sensibles et accès production.
- Patch management régulier des nœuds et images de base.
- WAF/rate limiting sur endpoints publics.

20. Infrastructure as Code et GitOps

L'infrastructure doit être reproductible. Terraform/OpenTofu gère ressources fournisseur, réseau, DNS et services managés. Helm/Kustomize décrit Kubernetes. Un outil GitOps tel qu'Argo CD ou Flux peut réconcilier automatiquement l'état du cluster avec Git.

```
infra/
├── terraform/
│   ├── modules/
│   │   ├── network/
│   │   ├── kubernetes/
│   │   ├── database/
│   │   ├── object-storage/
│   │   ├── dns/
│   │   └── environments/
│   │       ├── staging/
│   │       └── production/
│   ├── helm/
│   ├── k8s/
│   └── ansible/
│       ├── inventory/
│       ├── roles/
│       └── playbooks/
```

21. Gestion des dépendances externes

Les providers de paiement, SMS, email, maps/geocoding et push notifications sont considérés comme dépendances faillibles. Les timeouts sont courts, les retries bornés, les webhooks vérifiés cryptographiquement et les traitements découplés via Kafka lorsque la réponse immédiate n'est pas nécessaire.

22. Plan de déploiement initial

Phase	Livrables
P0 - Fondation	Git, registry, réseau, DNS, TLS, cluster, namespaces, RBAC
P1 - Data plane	PostgreSQL, Redis, Kafka, stockage objet, backups
P2 - Plateforme	Ingress, Gateway, secrets, observabilité, logging, tracing
P3 - Services communs	Identity, User, Payment, Notification, Wallet
P4 - Marketplace	Catalog, Seller, Inventory, Cart, Order
P5 - Food	Restaurant, Menu, Food Cart, Food Order
P6 - Delivery	Delivery, Dispatch, Driver, Tracking
P7 - Hardening	load tests, chaos tests ciblés, DR, sécurité, canary/rollback

23. Checklist avant mise en production

- DNS et certificats TLS valides.
- NetworkPolicies et RBAC testés.
- Secrets production injectés depuis le gestionnaire de secrets.
- Backups automatiques + restauration validée.
- Tous les services ont readiness/liveness/startup probes.
- Requests/limits et HPA configurés.
- Kafka replication, ISR, DLQ et alertes consumer lag configurés.
- DB migrations backward-compatible testées.
- Dashboards, logs et traces disponibles.
- Alertes critiques testées.
- Scans vulnérabilités sans criticité bloquante.
- Tests E2E checkout Marketplace et Food réussis.
- Flux paiement/webhooks testés.
- Flux Delivery/Dispatch testé.
- Procédure rollback exécutée en staging.
- Runbooks incident disponibles.

24. Runbooks minimum

Incident	Procédure requise
Pod CrashLoopBackOff	logs, events, config/secrets, rollback
Kafka consumer lag élevé	consumer health, partitions, scale-out, DLQ
DB saturation	pool, slow queries, locks, CPU/IO, scale
Paiement indisponible	circuit breaker, statut pending, reprise webhook
Nœud Kubernetes perdu	rescheduling, PDB, capacité restante
Certificat expirant	cert-manager, DNS challenge, renouvellement
Déploiement dégradé	stop rollout, rollback, analyse métriques
Perte DB primaire	promotion replica / restauration PITR

25. Critères d'acceptation

L'infrastructure est considérée prête pour production lorsque : (1) elle est reconstruisible à partir de l'IaC et Git ; (2) un déploiement complet staging est automatisé ; (3) la promotion production et le rollback sont reproductibles ; (4) les

communications gRPC et Kafka sont sécurisées et observables ; (5) les sauvegardes sont restaurables ; (6) aucun datastore interne n'est exposé publiquement ; (7) les SLO et alertes critiques sont mesurables ; (8) les procédures d'incident sont documentées et testées.